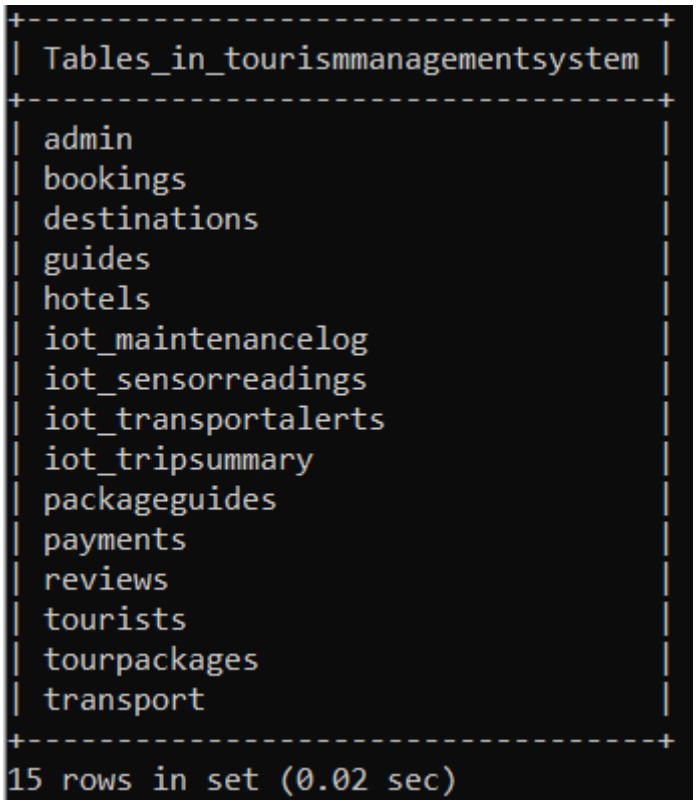
SQL Output 1: Database Tables Created



```
+-----------------------------------+
| Tables_in_tourismmanagementsystem |
+-----------------------------------+
| admin                             |
| bookings                          |
| destinations                      |
| guides                            |
| hotels                            |
| iot_maintenancelog                |
| iot_sensorreadings                |
| iot_transportalerts               |
| iot_tripsummary                   |
| packageguides                     |
| payments                          |
| reviews                           |
| tourists                          |
| tourpackages                      |
| transport                         |
+-----------------------------------+
15 rows in set (0.02 sec)
```

Figure 3 List of Tables Created in the Tourism Management System Database.

13. Database Verification and Display Queries

After creating the database, SQL commands were used to verify the structure and display records from all tables.

```
SHOW DATABASES;
USE TourismManagementSystem;
SHOW TABLES;

DESCRIBE Tourists;
DESCRIBE Destinations;
DESCRIBE Hotels;
DESCRIBE TourPackages;
DESCRIBE Bookings;
DESCRIBE Payments;
DESCRIBE Guides;
DESCRIBE PackageGuides;
DESCRIBE Reviews;
DESCRIBE Transport;
DESCRIBE IoT_SensorReadings;
DESCRIBE IoT_TransportAlerts;
DESCRIBE IoT_TripSummary;
DESCRIBE IoT_MaintenanceLog;

SELECT * FROM Tourists;
SELECT * FROM Destinations;
```

```
SELECT * FROM Hotels;
SELECT * FROM TourPackages;
SELECT * FROM Bookings;
SELECT * FROM Payments;
SELECT * FROM Guides;
SELECT * FROM PackageGuides;
SELECT * FROM Reviews;
SELECT * FROM Transport;
SELECT * FROM IoT_SensorReadings;
SELECT * FROM IoT_TransportAlerts;
SELECT * FROM IoT_TripSummary;
SELECT * FROM IoT_MaintenanceLog;
```

SQL Output 2: Structure of Tourists Table

```
+-----------------+--------------+------+-----+-------------------+-------------------+
| Field           | Type         | Null | Key | Default           | Extra             |
+-----------------+--------------+------+-----+-------------------+-------------------+
| tourist_id      | int          | NO   | PRI | NULL              | auto_increment    |
| full_name       | varchar(100) | NO   |     | NULL              |                   |
| gender          | varchar(10)  | YES  |     | NULL              |                   |
| date_of_birth   | date         | YES  |     | NULL              |                   |
| nationality     | varchar(50)  | YES  |     | NULL              |                   |
| email           | varchar(100) | NO   | UNI | NULL              |                   |
| phone           | varchar(20)  | YES  |     | NULL              |                   |
| passport_number | varchar(50)  | YES  |     | NULL              |                   |
| address         | varchar(255) | YES  |     | NULL              |                   |
| created_at      | timestamp    | YES  |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
+-----------------+--------------+------+-----+-------------------+-------------------+
10 rows in set (0.02 sec)
```

Figure 4 Structure of the Tourists Table.

SQL Output 3: Structure of Bookings Table

```
+------------------+---------------+------+-----+---------+----------------+
| Field            | Type          | Null | Key | Default | Extra          |
+------------------+---------------+------+-----+---------+----------------+
| booking_id       | int           | NO   | PRI | NULL    | auto_increment |
| tourist_id       | int           | YES  | MUL | NULL    |                |
| package_id       | int           | YES  | MUL | NULL    |                |
| booking_date     | date          | YES  |     | NULL    |                |
| number_of_people | int           | YES  |     | NULL    |                |
| total_amount     | decimal(10,2) | YES  |     | NULL    |                |
| booking_status   | varchar(30)   | YES  |     | Pending |                |
+------------------+---------------+------+-----+---------+----------------+
```

Figure 5 Structure of Bookings Table

SQL Output 4: Structure of Transport Table

```
+--------------------+---------------+------+-----+---------+----------------+
| Field              | Type          | Null | Key | Default | Extra          |
+--------------------+---------------+------+-----+---------+----------------+
| transport_id       | int           | NO   | PRI | NULL    | auto_increment |
| transport_type     | varchar(50)   | YES  |     | NULL    |                |
| company_name       | varchar(100)  | YES  |     | NULL    |                |
| departure_location | varchar(100)  | YES  |     | NULL    |                |
| arrival_location   | varchar(100)  | YES  |     | NULL    |                |
| departure_time     | datetime      | YES  |     | NULL    |                |
| arrival_time       | datetime      | YES  |     | NULL    |                |
| ticket_price       | decimal(10,2) | YES  |     | NULL    |                |
| iot_device_id      | varchar(50)   | YES  | UNI | NULL    |                |
| vehicle_number     | varchar(30)   | YES  |     | NULL    |                |
| iot_device_status  | varchar(20)   | YES  |     | Active  |                |
| iot_installed_date | date          | YES  |     | NULL    |                |
| last_ping_time     | datetime      | YES  |     | NULL    |                |
| is_iot_enabled     | tinyint(1)    | YES  |     | 1       |                |
+--------------------+---------------+------+-----+---------+----------------+
14 rows in set (0.01 sec)
```

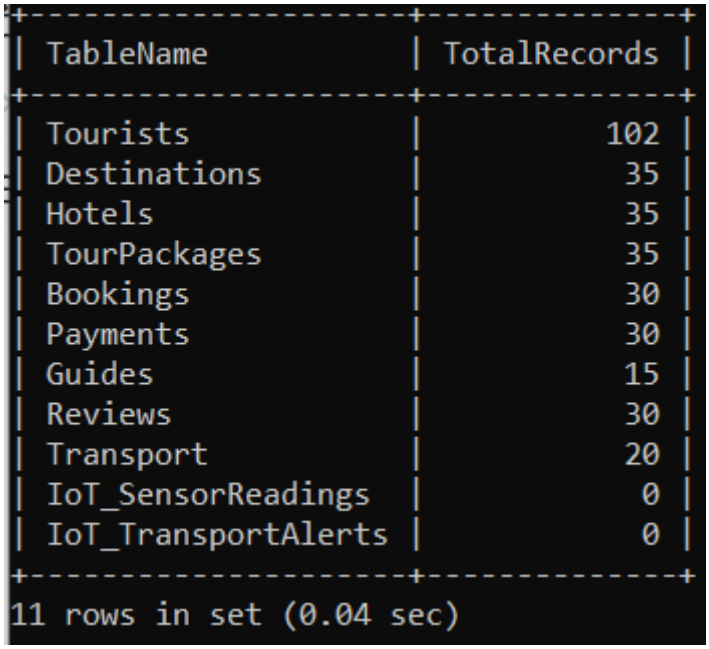
Figure 6 Structure of Transport Table

SQL Output 5: Structure of IoT Sensor Readings Table

```
+----------------------+---------------+------+-----+---------+----------------+
| Field                | Type          | Null | Key | Default | Extra          |
+----------------------+---------------+------+-----+---------+----------------+
| reading_id           | int           | NO   | PRI | NULL    | auto_increment |
| transport_id         | int           | NO   | MUL | NULL    |                |
| reading_timestamp    | datetime      | NO   |     | NULL    |                |
| gps_latitude         | decimal(10,7) | YES  |     | NULL    |                |
| gps_longitude        | decimal(10,7) | YES  |     | NULL    |                |
| gps_altitude_meters  | decimal(8,2)  | YES  |     | NULL    |                |
| gps_signal_strength  | varchar(10)   | YES  |     | NULL    |                |
| current_speed_kmh    | decimal(6,2)  | YES  |     | NULL    |                |
| max_allowed_speed    | decimal(6,2)  | YES  |     | NULL    |                |
| speed_status         | varchar(20)   | YES  |     | NULL    |                |
| cabin_temp_celsius   | decimal(5,2)  | YES  |     | NULL    |                |
| outside_temp_celsius | decimal(5,2)  | YES  |     | NULL    |                |
| engine_temp_celsius  | decimal(6,2)  | YES  |     | NULL    |                |
| fuel_level_percent   | decimal(5,2)  | YES  |     | NULL    |                |
| fuel_consumed_liters | decimal(7,2)  | YES  |     | NULL    |                |
| current_passengers   | int           | YES  |     | NULL    |                |
| max_capacity         | int           | YES  |     | NULL    |                |
| engine_status        | varchar(30)   | YES  |     | NULL    |                |
| engine_rpm           | int           | YES  |     | NULL    |                |
| battery_voltage      | decimal(5,2)  | YES  |     | NULL    |                |
| door_status          | varchar(20)   | YES  |     | NULL    |                |
+----------------------+---------------+------+-----+---------+----------------+
21 rows in set (0.00 sec)
```

Figure 7 Structure of IoT Sensor Readings Table

SQL Output 6: Total Records Stored in Main Tables



TableName	TotalRecords
Tourists	102
Destinations	35
Hotels	35
TourPackages	35
Bookings	30
Payments	30
Guides	15
Reviews	30
Transport	20
IoT_SensorReadings	0
IoT_TransportAlerts	0

11 rows in set (0.04 sec)

Figure 8 Total Records Stored in Main Tables

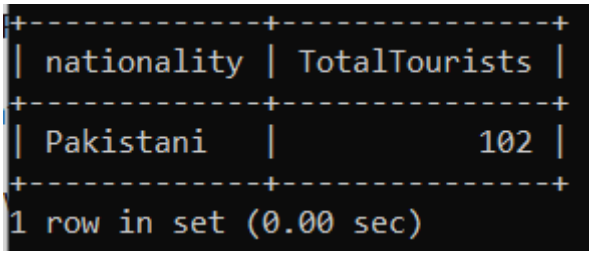
14. Important SQL Analysis Queries

Query 1: Tourist Nationality Distribution

This query shows how many tourists belong to each nationality. It helps to understand the main tourist groups in the system.

```
SELECT nationality,
COUNT(*) AS TotalTourists
FROM Tourists
GROUP BY nationality
ORDER BY TotalTourists DESC;
```

SQL Output 7: Tourist Nationality Distribution



nationality	TotalTourists
Pakistani	102

1 row in set (0.00 sec)

Figure 9 Tourist Nationality Distribution

Query 2: Most Booked Destinations

This query shows which destinations are booked the most. It joins destinations, tour packages, and bookings.

```
SELECT d.destination_name,
COUNT(b.booking_id) AS TotalBookings
FROM Destinations d
LEFT JOIN TourPackages tp ON d.destination_id = tp.destination_id
LEFT JOIN Bookings b ON tp.package_id = b.package_id
GROUP BY d.destination_name
ORDER BY TotalBookings DESC;
```

SQL Output 8: Most Booked Destinations

destination_name	TotalBookings
Fairy Meadows	1
Attabad Lake	1
Khunjerab Pass	1
Deosai Plains	1
Ratti Gali Lake	1
Saif ul Malook Lake	1
Babusar Top	1
Shogran	1

Figure 10 Most Booked Destinations

Maldives Beach Resort	1
Toli Pir	0
Makran Coastal Highway	0
Pakistan Monument	0
Rawal Lake	0
Minar e Pakistan	0

35 rows in set (0.00 sec)

Figure 11 Most Booked Destinations

Query 3: Most Popular Transport Routes

This query shows the most scheduled transport routes. It helps to find which routes are used more often.

```
SELECT departure_location,
arrival_location,
COUNT(*) AS TotalScheduledTrips
FROM Transport
GROUP BY departure_location, arrival_location
ORDER BY TotalScheduledTrips DESC;
```

SQL Output 10: Booking Status Summary

departure_location	arrival_location	TotalScheduledTrips
Karachi	Lahore	1
Lahore	Islamabad	1
Islamabad	Skardu	1
Karachi	Islamabad	1
Chilas	Fairy Meadows	1
Lahore	Multan	1
Islamabad	Gilgit	1
Gilgit	Hunza	1
Rawalpindi	Naran	1
Naran	Saif ul Malook	1
Islamabad	Quetta	1
Quetta	Gwadar	1
Karachi	Gwadar	1
Lahore	Rawalpindi	1
Islamabad	Dubai	1
Islamabad	Paris	1
Lahore	Tokyo	1
Islamabad	Male	1
Islamabad	Zurich	1
Peshawar	Kalam	1

20 rows in set (0.00 sec)

Figure 12 Booking Status Summary

Query 4: Booking Status Summary

This query shows the number of bookings in each booking status.

```
SELECT booking_status,
COUNT(*) AS TotalBookings
FROM Bookings
GROUP BY booking_status;
```

booking_status	TotalBookings
Confirmed	21
Pending	7
Cancelled	2

3 rows in set (0.00 sec)

Query 5: Revenue by Payment Method

This query shows total completed payment amount by payment method.

```
SELECT payment_method,
SUM(payment_amount) AS TotalRevenue
FROM Payments
WHERE payment_status = 'Completed'
GROUP BY payment_method
ORDER BY TotalRevenue DESC;
```

SQL Output 11: Revenue by Payment Method

```
+----------------+--------------+
| payment_method | TotalRevenue |
+----------------+--------------+
| Debit Card     |    667500.00 |
| Credit Card    |    320000.00 |
| Cash           |    298000.00 |
| JazzCash       |    237500.00 |
| Bank Transfer  |    220000.00 |
| EasyPaisa      |    191000.00 |
+----------------+--------------+
6 rows in set (0.191 sec)
```

Figure 13 Revenue by Payment Method

Query 6: IoT Transport Alerts by Severity

This query shows how many alerts exist in each severity level. It helps to understand transport safety issues.

```
SELECT alert_severity,
COUNT(*) AS TotalAlerts
FROM IoT_TransportAlerts
GROUP BY alert_severity
ORDER BY TotalAlerts DESC;
```

SQL Output 12: IoT Transport Alerts by Severity

```
+----------------+-------------+
| alert_severity | TotalAlerts |
+----------------+-------------+
| Medium         |           5 |
| High           |           5 |
| Low            |           2 |
| Critical       |           2 |
+----------------+-------------+
4 rows in set (0.627 sec)
```

Figure 14 IoT Transport Alerts by Severity

Query 7: Transport Utilization by Route

This query counts total trips for each route. It helps to analyze route utilization.

```
SELECT departure_location,
arrival_location,
COUNT(transport_id) AS TotalTrips
FROM Transport
```

```
GROUP BY departure_location, arrival_location
ORDER BY TotalTrips DESC;
```

SQL Output 13: Transport Utilization by Route

departure_location	arrival_location	TotalTrips
Karachi	Lahore	1
Lahore	Islamabad	1
Islamabad	Skardu	1
Karachi	Islamabad	1
Chilas	Fairy Meadows	1
Lahore	Multan	1
Islamabad	Gilgit	1
Gilgit	Hunza	1
Rawalpindi	Naran	1
Naran	Saif ul Malook	1
Islamabad	Quetta	1
Quetta	Gwadar	1
Karachi	Gwadar	1
Lahore	Rawalpindi	1
Islamabad	Dubai	1
Islamabad	Paris	1
Lahore	Tokyo	1
Islamabad	Male	1
Islamabad	Zurich	1
Peshawar	Kalam	1

20 rows in set (0.00 sec)

Figure 15 Transport Utilization by Route

15. Power BI Dashboard Objectives

Power BI was used to convert database records into visual insights. These charts make the data easier to understand. They also help management make better decisions.

Objective 1: Tourist Nationality Distribution

Purpose: To identify the distribution of tourists by nationality.

Recommended Visual: Clustered column chart or bar chart.

Power BI Fields:

X axis: nationality

Y axis: Count of tourist_id

Value name: TotalTourists

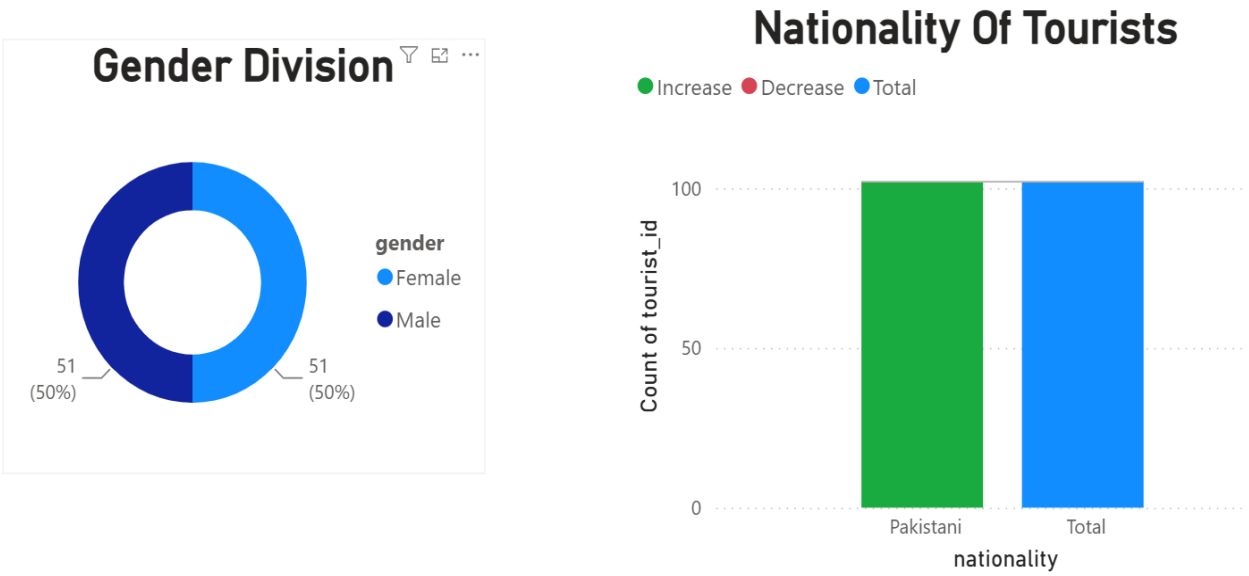


Figure 16 Tourist Nationality Distribution.

This chart shows how tourists are divided by nationality. It helps identify the countries or regions from where most tourists come. This information can support marketing, guide language planning, and service improvement.

Objective 1: Booking Status Analysis

The first Power BI objective is to analyze the booking status in the Tourism Management System. Booking status is important because it shows how many bookings are confirmed, pending, or cancelled. This helps the tourism management team understand the current condition of customer bookings.

The chart shows three booking categories. Confirmed bookings are the highest, with 21 bookings, which represent 70 percent of the total bookings. Pending bookings are 7, which represent 23.33 percent. Cancelled bookings are 2, which represent 6.67 percent.

This result shows that most customers have confirmed their bookings. This is a positive sign because it means the system has a strong number of completed booking decisions. The number of pending bookings is also important because these bookings still need follow up. The cancelled bookings are very low, which shows that only a small number of tourists cancelled their plans.

Overall, the booking status analysis helps in monitoring customer booking behavior. It also helps the tourism organization manage confirmed trips, follow pending bookings, and review cancelled bookings for better planning.

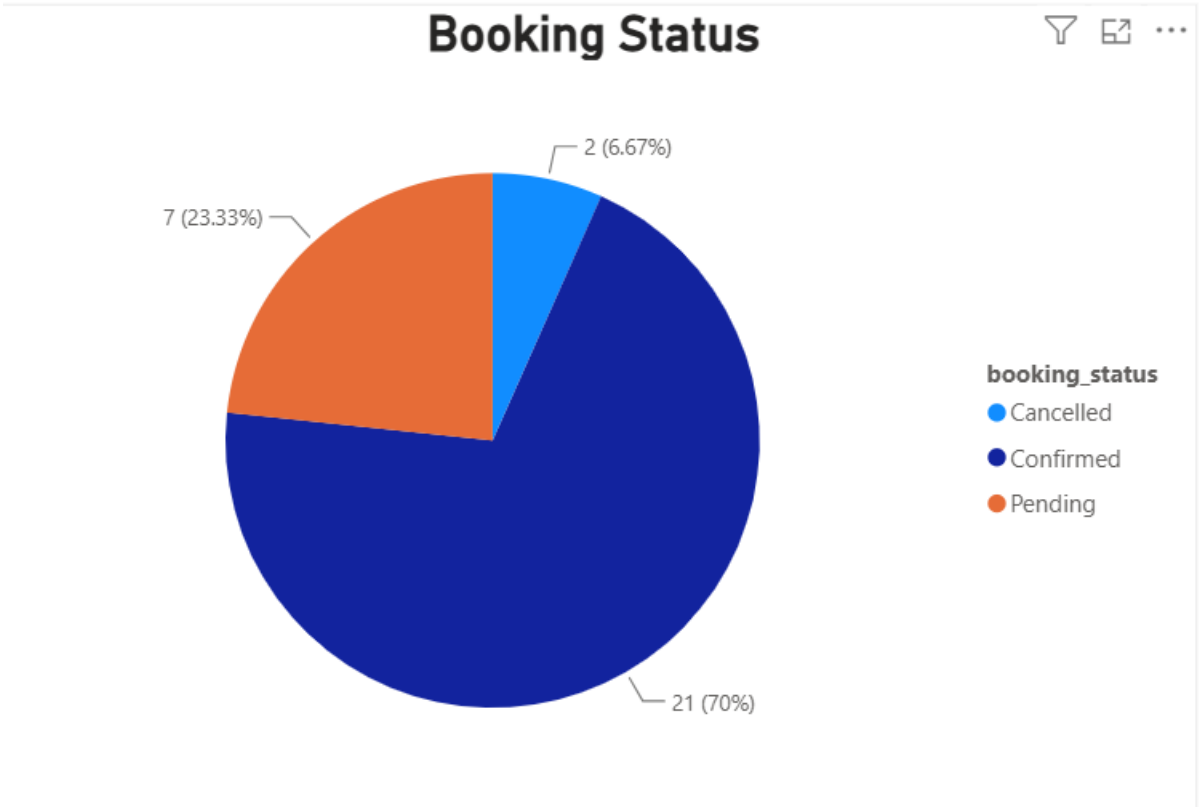


Figure 17 Booking Status Distribution

Objective 2: Payment Status Analysis

The second Power BI objective is to analyze the payment status in the Tourism Management System. Payment status is important because it shows how much payment amount is paid, pending, or refunded. This helps the tourism management team understand the financial condition of bookings.

The chart shows three payment categories. The paid amount is the highest, with approximately 3.9 million. The pending amount is approximately 1.2 million. The refunded amount is approximately 0.3 million.

This result shows that most of the payment amount has already been paid. This is a positive sign because it means a large portion of the booking revenue has been received. The pending amount still needs attention because it shows payments that are not completed yet. The refunded amount is lower, which means only a small portion of the total payment amount was returned.

Overall, the payment status analysis helps the tourism organization monitor revenue collection. It also helps identify pending payments and refunded amounts. This supports better financial tracking and booking management.

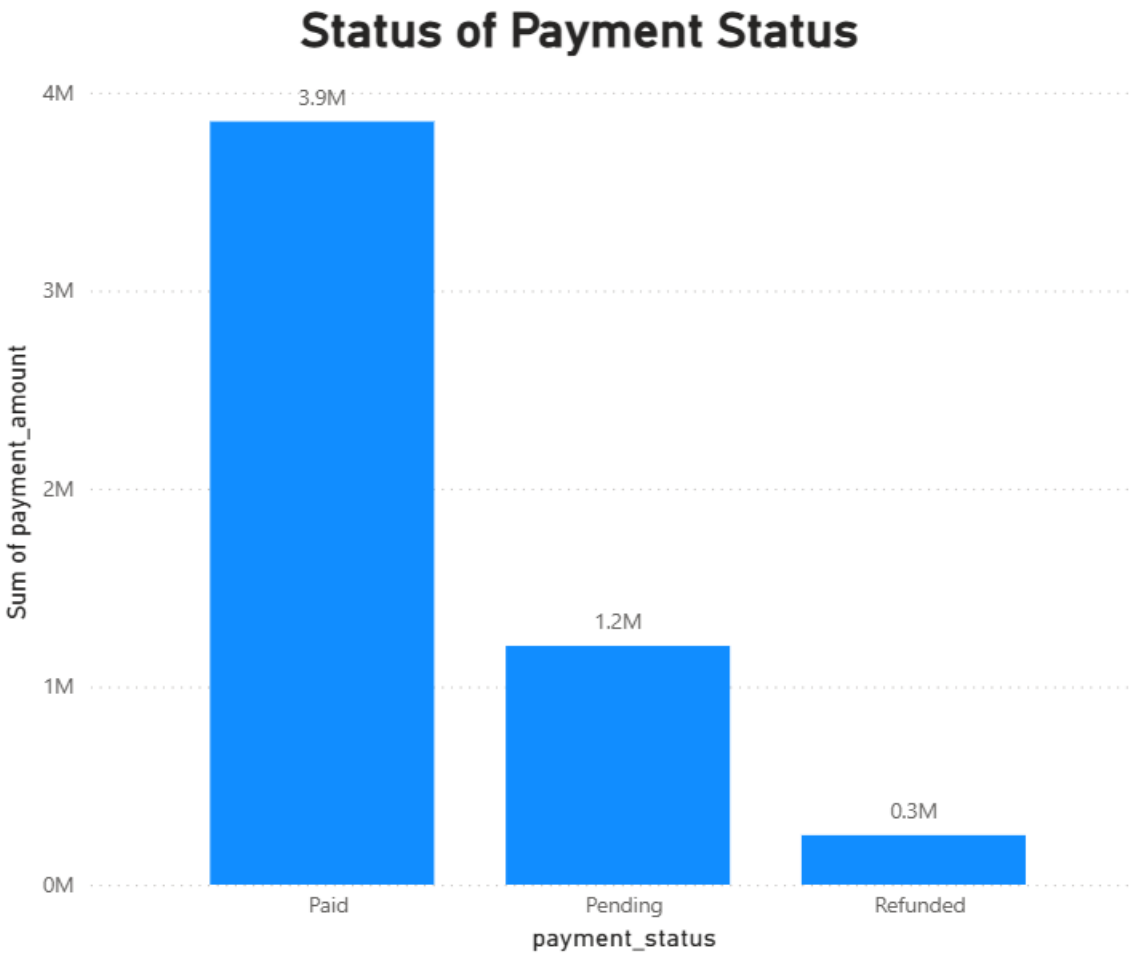


Figure 18 Payment Amount by Payment Status.

16. Discussion

The Tourism Management System database supports the main operations of a tourism organization. It stores tourist information, package details, destination records, booking data, payment records, guide assignments, reviews, and transport monitoring data in one structured system.

The ERD improves understanding of the database design. It clearly shows how the tables are connected. It also shows primary keys, foreign keys, and relationship cardinalities. This makes the database easier to understand and implement.

The EERD improves the design further. It adds advanced modelling concepts. Transport specialization makes the transport structure more flexible. GuideLanguages removes the multivalued language problem. PackageGuides solves the many to many relationship between packages and guides. IoT aggregation shows the monitoring process in a more meaningful way.

Normalization reduces duplicate data and improves consistency. It makes sure that data is stored in the correct table. It also helps avoid future errors when data is inserted, updated, or deleted.

Anomaly removal makes the system more reliable. It prevents duplicate guide assignments. It protects old booking amounts. It keeps payment records clear. It also ensures that trip summaries are generated in a controlled way.

The MySQL implementation confirms that the design can work as a real database. The verification queries help check the database structure. The analysis queries help extract useful results. Power BI then converts these results into charts. These charts support better decision making in tourism management.

17. Limitations

The project focuses on database design and analysis. It does not include a complete software application.

The IoT integration is represented at the database level only. Real time hardware connection is not included.

Online payment gateway integration is not included. The Payments table only stores payment records.

Power BI outputs depend on the sample data stored in the database. If the sample data is limited, the charts will also be limited.

Some derived fields need views, triggers, or generated columns for stronger accuracy. These include destination rating, speed status, average speed, and safety score.

18. Conclusion

This project successfully designed and implemented a Tourism Management System database with IoT based transport monitoring. The database provides a structured way to manage tourism records and transport monitoring records.

The ERD clearly represents the main entities and their relationships. The EERD improves the design by adding specialization, aggregation, associative entities, multivalued attribute handling, and derived attributes. Normalization improves data consistency and reduces repetition. Anomaly removal reduces common database problems such as insertion, update, deletion, duplication, and derived data issues.

The MySQL implementation confirms that the database can be created and used practically. The verification queries help check the tables. The analysis queries provide useful information from the database. Power BI visualizations make the data easier to understand and support better decision making.

Overall, this project provides a strong database foundation for a tourism management system. In the future, it can be extended with a web application, mobile application, live IoT sensors, online payment gateway, and advanced dashboard features.